

Synchronization Schemes for a Balanced Binary Search Tree

CPEG 655 – High Performance Computing Project

Topic 2: Synchronization Schemes for Concurrent BST

Student: Rajeev Reddy

Semester: Fall 2025

1. Introduction

Binary search trees (BSTs) are a fundamental data structure for ordered key-value lookup. On a single core, a well-implemented BST can handle millions of searches per second. On multicore CPUs, however, concurrency and synchronization become the dominant challenges. If multiple threads traverse or update a shared tree without synchronization, we risk race conditions and corrupted data. If we over-synchronize with a single global lock, we sacrifice scalability and underutilize available cores.

This project investigates synchronization schemes for a balanced binary search tree under a read-only workload. Building on the Lab 3 concurrent BST assignment, I implement and evaluate several schemes on the same balanced structure:

- seq – sequential baseline (single-threaded, no locks)
- cg – coarse-grain global lock
- fg – fine-grain path-based locking
- ideal – read-only, no-lock multi-thread mode

The goal is to understand how much performance is lost to synchronization, independent of pathologically-shaped trees, and to compare the observed scaling against an upper bound provided by the ideal no-lock mode.

2. Background and Motivation

In Lab 3, we implemented concurrent search on a binary search tree where the structure was determined by insertion order. Depending on the key distribution, the tree could be nearly balanced or badly skewed. This made performance analysis ambiguous: poor throughput could stem from either synchronization overhead or simply long search paths in an unbalanced tree.

In this project (Topic 2), the focus shifts from arbitrary trees to a deliberately balanced BST. Using a helper routine `bst_build_balanced`, the tree is constructed from a sorted array of keys, guaranteeing logarithmic height and relatively uniform path lengths. With tree-shape variability removed, we can attribute differences in throughput primarily to the synchronization mechanisms themselves.

The project therefore provides a cleaner environment to answer two questions:

- 1) How do coarse-grain and fine-grain locking schemes behave on a balanced tree under a read-only workload?
- 2) How close can we get to an idealized no-lock upper bound on throughput on our multicore hardware?

3. Design and Implementation

3.1 Data Structure: Balanced BST

The core data structure is a classic pointer-based binary search tree with integer keys. The helper function `bst_build_balanced` constructs a balanced tree from a sorted array by recursively selecting the midpoint as the root of each subtree. The result is a tree of height $O(\log N)$ with relatively uniform root-to-leaf path lengths.

Once the balanced tree is built, it is treated as read-only for the experiments in this report. All operations are search-only; no insertions or deletions occur during timing runs. This simplifies correctness and isolates the cost of synchronization on traversals.

3.2 Sequential Baseline (seq)

The sequential baseline, `seq`, uses a single thread and performs standard BST search without any locks. Given a key, the algorithm compares it with the current node, descends left or right accordingly, and terminates at a matching node or a null pointer. This implementation is straightforward and serves as the reference for both throughput and speedup calculations.

Because `seq` is strictly single-threaded, its throughput is expected to be constant across experiments; any variation arises only from measurement noise.

3.3 Coarse-Grain Locking (cg)

The coarse-grain scheme, `cg`, protects the entire tree with a single global mutex. Each search operation executes the following pattern:

- 1) Acquire the global lock.
- 2) Traverse the balanced BST using the same search code as in `seq`.
- 3) Release the global lock.

This approach is trivial to implement and guarantees correctness under concurrent access. However, contention on the single mutex becomes the dominant cost as thread count increases, effectively serializing the workload and limiting scalability.

3.4 Fine-Grain Path Locking (fg)

The fine-grain scheme, `fg`, distributes locks along the search path through the tree. Each node (or small group of nodes) is associated with its own lock. A searching thread acquires locks while traversing from the root to a leaf and releases them as it moves down the tree.

In principle, fine-grain locking allows different threads to traverse disjoint subtrees concurrently, reducing contention compared to a global lock. In practice, upper levels of the tree (especially the root) remain hot spots that many threads must pass through, and each search pays the overhead of multiple lock and unlock operations. The net effect is that `fg`

often improves over cg but still incurs substantial synchronization cost on a read-only workload.

3.5 Ideal No-Lock Read-Only Mode

To estimate an upper bound on achievable throughput, an ideal mode is implemented for read-only workloads. In this mode, the balanced BST is treated as immutable and multiple threads are allowed to traverse the tree without any synchronization. Because no updates occur during timing runs, the tree structure does not change and concurrent reads are safe.

This ideal mode is not a complete solution for general concurrent BST usage, as it does not handle updates. However, it exposes the capability of the hardware and the balanced tree when synchronization overhead is removed. The gap between ideal and the locking schemes quantifies how expensive synchronization is on this platform.

4. Experimental Methodology

All experiments are conducted on an Intel Core i9 laptop CPU running a Linux environment. The benchmark harness accepts parameters for initial tree size, number of search operations, tree type, synchronization mode, and thread count. For the results reported here, the configuration is:

- Tree type: balanced (constructed with `bst_build_balanced`)
- `N_INIT` = 100,000 keys
- `N_SEARCH` = 5,000,000 search operations
- Workload: read-only searches (no inserts or deletes)
- Modes: seq, cg, fg, ideal
- Threads: $T \in \{1, 8, 32\}$ (representative points from the full sweep)

Each experiment measures total elapsed wall-clock time for the batch of searches.

Throughput is computed as:

$$\text{throughput} = \text{N_SEARCH} / \text{elapsed_time}$$

and is reported in millions of operations per second (M ops/s). For seq, the implementation is inherently single-threaded and ignores the thread-count parameter; runs at higher nominal T therefore correspond to repeated single-thread measurements and serve as a flat baseline.

5. Results

5.1 Measured Throughput on Balanced BST

Table 1 summarizes measured elapsed time and throughput for the balanced BST under different synchronization schemes and thread counts. These data points correspond to the plots used in the presentation. Figures should be inserted here in the final version of the report:

[Figure 1: Throughput vs. Threads (balanced tree, seq vs cg vs fg vs ideal)]

[Figure 2: Bar chart of throughput at T = 1, 8, 32 for all modes]

Threads (T)	Mode	Elapsed time (s)	Throughput (M ops/s)	Notes
1	seq	0.042833	116.7	Sequential baseline
1	cg	0.074106	67.5	Coarse-grain global lock
1	fg	0.071887	69.6	Fine-grain path locks
1	ideal	0.047826	104.5	Read-only, no locks
8	seq	0.042821	116.8	Single-thread baseline
8	cg	0.591972	8.4	Heavy contention on global lock
8	fg	0.174402	28.7	Better than cg, still lock-limited
8	ideal	0.014604	342.4	Scales well with threads
32	seq	0.043361	115.3	Single-thread baseline
32	cg	0.806205	6.2	Severe contention, throughput collapse
32	fg	0.293408	17.0	Lock overhead + shared hot spots
32	ideal	0.004774	1047.3	~ 10× speedup over ideal T=1

Table 1: Measured elapsed time and throughput for balanced BST under different synchronization schemes.

5.2 Analysis of Synchronization Schemes

Several trends emerge from Table 1 and the corresponding throughput plots:

- Sequential baseline (seq): The seq mode sustains approximately 116–117 M ops/s and remains flat across thread counts. This is expected because the implementation is single-threaded and does not benefit from additional cores.
- Coarse-grain locking (cg): cg already starts below seq at $T = 1$ due to the cost of acquiring and releasing the global mutex around every search. As T increases, threads spend most of their time waiting on the single lock. Throughput collapses to 8.4 M ops/s at $T = 8$ and 6.2 M ops/s at $T = 32$, which is more than an order of magnitude slower than seq.
- Fine-grain locking (fg): fg distributes locks along the search path and allows more parallelism than cg. However, every search still passes through the root and upper levels of the tree, creating shared hot spots. In addition, each search incurs multiple lock/unlock operations. Throughput improves over cg but never surpasses even the single-thread seq baseline; at $T = 32$ it reaches only 17.0 M ops/s.
- Ideal no-lock mode (ideal): The ideal mode tracks the seq baseline closely at $T = 1$, confirming that the core search algorithm is efficient. When threads are added, ideal shows strong scaling, reaching 342.4 M ops/s at $T = 8$ and 1047.3 M ops/s at $T = 32$. This corresponds to about a $10\times$ speedup over ideal $T = 1$ and roughly a $9\times$ speedup over the seq baseline.

Overall, the balanced tree and hardware clearly support high throughput; the primary obstacle to scaling is synchronization.

6. Discussion

The experiments on a balanced BST isolate synchronization as the dominant bottleneck. When tree shape is well-behaved and workloads are read-only, there is little inherent work per search—just a bounded number of pointer dereferences and comparisons. In this regime, adding expensive synchronization around each search quickly overwhelms the useful work.

Coarse-grain locking provides the simplest correctness story but effectively serializes the workload. Fine-grain locking improves the situation somewhat, yet still suffers from contention at the root and upper levels and from the cumulative cost of many small lock operations. The ideal mode demonstrates that, for read-only search on a balanced tree, the hardware can deliver close to an order of magnitude speedup when synchronization is removed.

These results suggest that for read-heavy workloads on mostly-immutable trees, alternative designs like read-copy-update, epoch-based reclamation, or lock-free search structures may be more appropriate than classic coarse- or fine-grain mutex schemes.

7. Limitations and Future Work

This project focuses on a read-only workload on a pre-built balanced BST. While this setting is useful for isolating synchronization costs, it does not capture all aspects of real applications:

- No updates: Insertions and deletions are not modeled, so contention patterns and correctness challenges for writers are not explored.
- Single data structure and workload: Only one tree size ($N_{\text{INIT}} = 100k$) and one search count ($N_{\text{SEARCH}} = 5M$) are evaluated. A more complete study would sweep over tree sizes, key distributions, and read/write ratios.
- Limited metrics: The primary metric here is throughput. Hardware counter data (e.g., cache misses, branch mispredictions) would give deeper insight into why particular schemes behave as they do.

Future work could extend the benchmark harness to support mixed read/write workloads, add additional synchronization schemes (such as read-write locks or lock-free trees), and integrate PAPI-based measurements to correlate high-level throughput with low-level microarchitectural behavior.

8. Conclusion

By moving from an arbitrary to a balanced BST and introducing an ideal no-lock read-only mode, this project separates the effects of tree shape from the costs of synchronization. The sequential baseline demonstrates that a single core can already sustain over 100 million searches per second on the balanced tree. Coarse-grain and fine-grain locking both fail to scale and, at high thread counts, perform an order of magnitude worse than the single-thread baseline. In contrast, the ideal mode shows strong scaling up to 32 threads, achieving roughly a 10× speedup over its own single-thread throughput.

The key takeaway is that on modern multicore CPUs, synchronization strategy can dominate performance even when the underlying data structure is well designed. Choosing or designing synchronization mechanisms that minimize contention and per-operation overhead is essential for exploiting available parallelism.

References

- [1] CPEG 655 Course Staff, "Lab 3: Concurrent Binary Search Tree," course handout.
- [2] CPEG 655 Course Staff, "Project Topic 2: Synchronization Schemes for Concurrent BST,"

project description.

[3] J. L. Hennessy and D. A. Patterson, Computer Architecture: A Quantitative Approach, 6th ed.